

Architecture Decision: Vector Search & Vertex AI Migration

Subject: Similarity Metric Rationale & Vertex AI Production Migration Path

As we transition the Teleport RAG Engine from local development to production, it is crucial that our local architecture mathematically aligns with our target cloud environment. This document outlines our choice of similarity metric and provides a direct, logical migration path to Google Cloud's Vertex AI.

1. Similarity Metric Rationale: Cosine vs. Euclidean (L2)

In our local FAISS implementation, we explicitly chose Cosine Similarity (achieved by L2-normalizing vectors and using an Inner Product index) over Euclidean (L2) distance.

- **Semantic Focus over Scale:** Cosine similarity measures the angle between vectors, entirely ignoring their magnitude. In semantic embedding spaces, the "direction" encodes the meaning, while the magnitude can be arbitrarily skewed by the length of the raw text. Cosine ensures we match purely on intent.
- **Target Model Alignment:** We are simulating Google's textembedding-gecko model. Google explicitly recommends Cosine similarity for semantic retrieval with this model family. By aligning our local math with the target production model, we prevent critical accuracy regressions during migration.
- **Bounded Confidence Scores:** Cosine yields strict scores between -1 and +1. This allows us to set intuitive, absolute confidence thresholds in our retrieval pipeline (e.g., "reject results below 0.65"). Euclidean distance has no natural upper bound, making it highly unpredictable for hard thresholding.
- **CPU Performance:** Because we normalize the embeddings upfront, Cosine similarity simplifies to a Dot Product. FAISS computes this using highly optimized SIMD CPU instructions, making our local CI/CD testing incredibly fast.

2. Architectural Patterns & Structural Design

To ensure the codebase is robust, testable, and production-ready, we architected the system using specific software engineering patterns:

- **Separation of Concerns (Modular Design):** The engine is strictly divided into decoupled modules:

Architecture Decision: Vector Search & Vertex AI Migration

- ``embeddings``, ``storage``, ``query_expansion``, and ``pipeline``. This means changes to the vector store (e.g., FAISS to Chroma) or the embedder (local to Vertex AI) do not bleed into the core orchestration logic.
- Interface Segregation & Duck Typing: The pipeline interacts with the embedder and storage layers purely through their method signatures (e.g., ``embed_batch``, ``search``). This guarantees that as long as a class fulfills the contract, it can be seamlessly swapped in without refactoring the orchestrator.
- Dependency Injection: The ``RAGPipeline`` class does not instantiate its own dependencies. Instead, the embedder, vector store, and query expander are passed (injected) into the pipeline upon initialization. This makes the pipeline incredibly easy to unit test (using our GCP mocks) and allows for dynamic configuration based on the environment (Dev vs. Prod).

3. Production Migration to Vertex AI Vector Search

While FAISS is excellent for testing, production requires high availability, automated scaling, and managed persistence. Thanks to our architectural design, here is our straightforward, zero-friction path to migrate to Vertex AI Vector Search (Matching Engine):

Step 1: Hot-Swap the Embedder Layer

Because we heavily rely on interface segregation, we will simply swap the current ``LocalEmbedder`` with our pre-built ``VertexAIEmbedder``. Because both expose the exact same Python interface, the ``RAG Pipeline`` will seamlessly begin routing requests to the live ``textembedding-gecko`` endpoint instead of the local sentence-transformer.

Step 2: Provision Vertex AI Infrastructure

We will provision a managed Vertex AI Vector Search Index configured to use the DOT_PRODUCT distance metric. Because our gecko embeddings are normalized, DOT_PRODUCT perfectly replicates the Cosine similarity behavior we tested locally. This index will be deployed to an Index Endpoint for low-latency gRPC querying.

Step 3: Implement VertexVectorStore

We will introduce a ``VertexVectorStore`` class conforming to our existing storage interface. Instead of interacting with a local FAISS file, this class will format batch embeddings into JSONL, upload them to a Google Cloud Storage (GCS) bucket to trigger an index update, and query the live endpoint for real-time retrieval.

Architecture Decision: Vector Search & Vertex AI Migration

Conclusion:

By adhering to strict architectural patterns like Dependency Injection and Separation of Concerns, we have completely de-risked our deployment. We can build and test complex retrieval logic locally at zero cost, confident that migrating to Vertex AI is simply a matter of swapping injected dependencies and providing GCP credentials.

By: Manish Singh Mehra